



RELATÓRIO TÉCNICO DE ELABORAÇÃO DE PROJETO

Sistema de Gestão de Estudantes em Linguagem C

AUTORES:

Laurindo Augusto Domingos

Adelina Salassambila

Garcia Monteiro

Kiaku Alberto

Figueiredo Npaca

Junho de 2026

1. Introdução

O presente relatório técnico descreve de forma detalhada o desenvolvimento e a arquitetura do Sistema de Gestão de Estudantes, concebido e implementado na linguagem de programação C. O sistema foi estruturado para atuar como uma solução robusta e modularizada, voltada para o armazenamento temporário, manipulação, pesquisa, ordenação e persistência de dados acadêmicos baseados em registros de estudantes.

A relevância prática deste projeto reside na aplicação direta de conceitos fundamentais das Estruturas de Dados e Algoritmos, tais como a manipulação de ficheiros (I/O), a alocação estática de memória baseada em vetores de estruturas (structs), e o uso de múltiplos métodos consagrados de ordenação e busca linear e binária. Através de um menu dinâmico orientado por consola, o utilizador obtém o controle operacional completo (CRUD) sobre a base de dados em memória.

2. Funcionalidades do Sistema

O software foi projetado para oferecer uma gama completa de operações de gerência de dados. Abaixo estão detalhadas todas as funcionalidades disponibilizadas através do menu interativo:

- **Leitura de Arquivos de Registos (Opção 1):** Permite carregar para a memória RAM os dados armazenados num ficheiro de texto estruturado (ex: 'estudantes.txt'). Os dados são convertidos de forma automatizada em elementos da estrutura do programa, efetuando o cálculo imediato da média aritmética de cada estudante.
- **Exibição da Lista Formatada (Opção 2):** Renderiza no terminal uma tabela alinhada e de fácil leitura contendo todas as variáveis de cada estudante ativo (Matrícula, Nome, Notas N1, N2, N3 e a Média com duas casas decimais).
- **Módulos Interativos de Ordenação (Opção 3):** Abre um submenu que oferece ao utilizador a escolha entre três algoritmos diferentes de ordenação aplicados sobre três chaves de ordenação distintas.
- **Módulos Interativos de Pesquisa (Opção 4):** Provê buscas exatas de registos tanto por identificador numérico único (Matrícula) quanto por correspondência alfabética (Nome).
- **Persistência e Salvaguarda (Opção 5):** Transfere com segurança todos os dados modificados ou inseridos na memória RAM de volta para o disco rígido, permitindo que o utilizador escolha exportar no formato padrão de Texto (.txt) ou no formato codificado Binário (.bin).
- **Inserção Manual via Teclado (Opção 6):** Permite adicionar um novo estudante em tempo de execução, coletando os dados campo a campo com proteções de buffer.

- **Edição Dinâmica de Registos (Opção 7):** Localiza um estudante através da matrícula e disponibiliza um menu de alteração seletiva (atualizar apenas o nome, apenas uma nota específica ou todo o registo).
- **Remoção e Eliminação de Dados (Opção 8):** Realiza a exclusão lógica e física em memória de um determinado estudante, reorganizando o vetor para evitar lacunas vazias.

3. Modelagem e Estrutura de Dados

A base do sistema assenta sobre a criação de um tipo de dados customizado chamado Estudante através da palavra-chave struct. O arranjo em memória é gerido por um vetor estático limitado por diretivas de pré-processamento que controlam a segurança do sistema:

```
#define MAX_ESTUDANTES 200
#define MAX_NOME 100
```

```
typedef struct Estudante {
    int matricula;
    char nome[MAX_NOME];
    float nota1;
    float nota2;
    float nota3;
    float media;
} Estudante;
```

Campos da Estrutura:

- **matricula (int):** Código de identificação exclusivo do aluno.
- **nome (char[]):** String que armazena o nome completo do estudante limitado a 99 caracteres mais o terminador nulo.
- **nota1, nota2, nota3 (float):** Avaliações contínuas obtidas pelo aluno.
- **media (float):** Campo calculado automaticamente em tempo de execução através da média aritmética das três notas.

4. Detalhamento Detalhado das Funções

O código segue os padrões modernos de design modular em C, isolando a lógica de negócios e as assinaturas em ficheiros de cabeçalho (funcoes.h) e as implementações em módulos de funções separados. A tabela a seguir descreve minuciosamente o papel de cada função presente no sistema:

Protótipo / Função	Parâmetros Recebidos	Descrição Detalhada do Funcionamento
void lerFicheiro(Estudiante lista[], int *total)	lista[]: Vetor de dados total: Ponteiro p/ contador global	Solicita o nome do arquivo .txt, realiza a abertura em modo leitura ('r'), valida sua existência e realiza o parse de dados usando fscanf com delimitadores ';'. Calcula a média e incrementa o total por referência.
void exibirEstudantes(Estudiante lista[], int total)	lista[]: Vetor de dados total: Inteiro por valor	Verifica se há registos em memória. Caso haja, imprime um cabeçalho formatado e percorre o vetor exibindo os dados de forma tabular utilizando formatação de largura fixa (%-25s, %-6.2f).
int comparar(Estudiante a, Estudiante b, int criterio)	a, b: Duas structs do tipo Estudiante criterio: Código (1, 2 ou 3)	Centraliza a lógica de comparação para os algoritmos de ordenação. Retorna verdadeiro se 'a > b' (para Matrícula ou Nome via strcmp) ou se 'a < b' (para Média Decrescente).
void bubbleSort(Estudiante lista[], int total)	lista[]: Vetor de dados total: Total de alunos criterio: Critério escolhido	Ordena a lista através de permutações adjacentes sucessivas (Bubble Sort).

int criterio)		Utiliza a função comparar() interna para determinar a troca dos elementos.
void selectionSort(Estudante int int criterio)	lista[]: Vetor de dados lista[], total: Total de alunos total, criterio: Critério escolhido	Percorre o vetor buscando o menor/maior elemento baseado no critério e realiza a troca direta com a posição inicial do loop corrente, otimizando o número de trocas.
void insertionSort(Estudante int int criterio)	lista[]: Vetor de dados lista[], total: Total de alunos total, criterio: Critério escolhido	Realiza a ordenação por inserção imediata, deslocando elementos maiores para a direita e inserindo a 'chave' na posição correta, simulando a organização de cartas.
void menuOrdenacao(Estudante int total)	lista[]: Vetor de dados lista[], total: Total de alunos	Exibe interface gráfica via terminal que captura o algoritmo pretendido e o critério desejado, invocando a rotina correspondente de ordenação.
int pesquisaLinearMatricula(Estudante int int matricula)	lista[]: Vetor total: Total de alunos matricula: Chave numérica total,	Realiza uma busca sequencial simples no vetor. Retorna o índice do elemento encontrado ou -1 caso não exista na base.
int pesquisaBinariaMatricula(Estudante int int matricula)	lista[]: Vetor total: Total de alunos matricula: Chave numérica lista[], total,	Divide o vetor ordenado consecutivamente ao meio para encontrar a matrícula. Reduz drasticamente a complexidade temporal para $O(\log N)$. Requer ordenação prévia.

int pesquisaLinearNome(Estudante int char nomeProcurado[])	lista[]: Vetor total: Total de alunos nomeProcurado: String	Busca sequencial utilizando a função strcasecmp para ignorar variações de caixa alta/baixa entre as strings dos nomes.
void menuPesquisa(Estudante int total)	lista[]: Vetor de dados total: Total de alunos	Interface que gerencia as opções de busca, solicita as entradas do utilizador e apresenta na tela os dados completos do aluno caso localizado.
void gravarFicheiro(Estudante int total)	lista[]: Vetor de dados total: Total de alunos	Permite salvar os dados. Em modo Texto (.txt) grava os dados sem a média para manter a compatibilidade de releitura. Em modo Binário (.bin), despeja a memória diretamente via fwrite.
void inserirEstudanteTeclado(Estudante int *total)	lista[]: Vetor total: Ponteiro para o contador	Valida o estouro do limite máximo, recebe via teclado os dados, realiza a limpeza preventiva do buffer e insere no fim do vetor, incrementando o contador por referência.
void editarEstudante(Estudante int total)	lista[]: Vetor de dados total: Total de alunos	Localiza o índice do aluno via pesquisa linear de matrícula, abre submenu para selecionar o campo de modificação e recalcula a média final pós-alteração.
void eliminarEstudante(Estudante int *total)	lista[]: Vetor total: Ponteiro para o contador	Localiza o registo, solicita uma confirmação ao utilizador e, caso positiva,

move todos os elementos subsequentes uma posição para a esquerda, decrementando o total.

5. Análise de Conclusão

A implementação do Sistema de Gestão de Estudantes atingiu todos os requisitos funcionais e estruturais previstos. A divisão clara de responsabilidades através de protótipos de funções garante que o sistema possa ser facilmente expandido no futuro (por exemplo, adicionando uma alocação dinâmica de memória ou uma interface gráfica).

Ademais, a inclusão de múltiplos métodos de ordenação e busca solidifica o projeto como uma excelente ferramenta de estudo comparativo de desempenho e lógica computacional, demonstrando de forma prática o impacto das escolhas de algoritmos no ciclo de vida de um software.